

Inigo Quilez :: articles :: filtering procedural textures - 2013

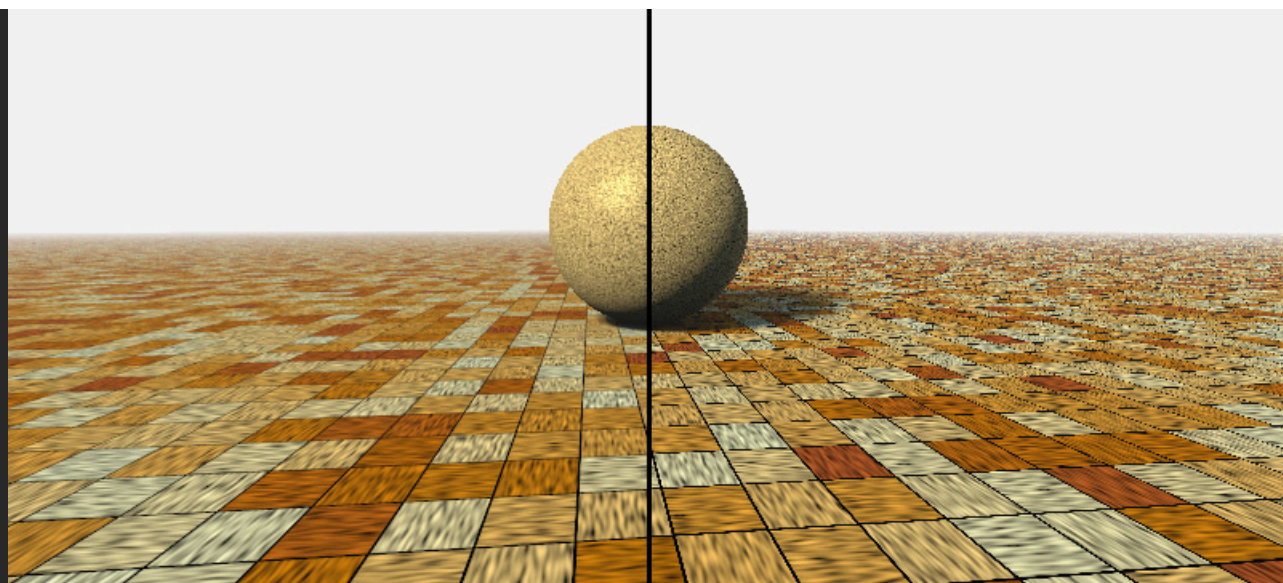


Intro

Procedural texturing/shading is a powerful tool in computer graphics. Its low memory requirements without tiling, and its natural ability to adapt to the geometry make it very attractive for many applications. However, unlike bitmap based texture approaches where aliasing is easily avoided by filtering through mipmaps, procedural patterns are difficult to antialias, unless backed into bitmap textures (sometimes on the fly at render time), which in some cases sort of defeats the point of designing patterns procedurally in the first place.

An often chosen alternative is to artificially clamp the amount of detail (high frequency content) of the procedural patterns, adaptively, in order to avoid aliasing (which happens when details in the pattern are higher frequency than half the sampling rate - see Nyquist). However, this approach is not convenient in terms of visual realism - for we usually want our shaders/patterns to be sharp and full of detail. Plus it's often very difficult to debug and one more concent more for the procedural artists what shouldn't be there - the artist should just focus on the pattern, not in manually antialias it.

This article is about a simple way to implement filtering/antialiasing of procedural pattern that doesn't require such manual detail clamping, nor backing the patterns into bitmap textures, which basically is a brute force approach. This method has been used before of course, but it doesn't seem to be very popular. However, I have found it to behave very well in practice, and these days of complex illumination models where lighting is an expensive part of the rendering process, this filtering method seems to shine.



Left side uses procedural texture filtering, right half doesn't

The filtering

The obvious solution is to supersample. The only thing one has to be careful about is doing it properly, ie, adaptively - don't oversample, don't undersample. Luckily for us, the problem is an old one in computer graphics and was solved for us long time ago: filter footprints. This is the technique GPUs use to pick the right mipmap level of a texture when accessing texels in a shader. It's the same technique raytracing based renderers use when employing ray differentials in their ray-casting process. In the end, the idea is that for a given image pixel that we are rendering to, we need to know how much area of a texture or pattern it covers.

When using a bitmap texture, this question can be rephrased as "how many texels of our texture do fall under this pixel". And when answered, one has to take indeed all those texels and average them to a single color, for the pixel can store only one color. That averaging or integration is usually precomputed at different texel counts in the so called mipmaps, so that the integration, which is obviously an expensive operation if done naively, can be skipped altogether at render time. Indeed, look-up tables (LUTs) are still useful these days.

With procedural patterns which don't undergo a caching/bitmap-baking process, since we don't have precomputed texels (which is a good thing and the point of the using them in the first place), such preintegration/precomputation cannot be performed. So we have to delegate the integration until pattern/shading generation time - which is render time. As for the filter width computation, that doesn't change between bitmap textures and procedural patterns.

So, let's focus on the filtering first, and the filter footprint computation later. Say we do have a procedural

pattern/texture called *sampleTexture()* like this:

```
// sample a procedural pattern
vec3 sampleTexture( in vec3 uvw );
```

which we have simplified to take a 3D texture coordinate only (think solid procedural texturing), although this might work equally well with 2D uv-based texture coordinates or other type of richer domain (such as one including normals, time, or whatever). The function takes the texture coordinates, runs some procedural code/algorithm/formula, and outputs a color for the surface to be shaded (such function could in fact return a full BRDF parametrization such as specular roughness, transparency, and other attributes, but lets simplify it to a diffuse color for now, if only to have something parallel to what you have when indexing into a simple bitmap texture). Then, an integrated/filtered lookup into such pattern in a given *uvw* domain is something as simple as a double loop evaluating the procedural pattern across the domain of interest:

```
// sample a procedural pattern with filtering
vec3 sampleTextureWithFilter( in vec3 uvw, in vec3 uvwX, in vec3 uvwY, in float detail )
{
    int sx = 1 + iclamp( int( detail*length(uvwX-uvw) ), 0, MaxSamples-1 );
    int sy = 1 + iclamp( int( detail*length(uvwY-uvw) ), 0, MaxSamples-1 );

    vec3 no = vec3( 0.0 );

    for( int j=0; j<sy; j++ )
    for( int i=0; i<sx; i++ )
    {
        vec2 st = vec2( float(i), float(j) )/vec2(float(sx),float(sy));
        no += sampleTexture( uvw + st.x * (ddx_uvw-uvw) + st.y*(ddy_uvw-uvw) );
    }

    return no / float(sx*sy);
}
```

Of course one can go fancy here and do actually a jittered grid-sampling to get some stratification. It's not the point of this article to show you how to sample a pattern. However, this pure grid-based approach works just beautifully.

Computing the filter footprint in a {ray/path}{tracer/marcher}

For a ray based renderers (meaning raytracing, raymarching, pathtracing, raycasting, pathmarching, etc, you name it whatever you want - except for *spheretracing* please, that's not what it is), one needs to be able to

compute ray/pixel footprints. One way to do it is to give the ray some thickness and make it a cone (as in cone tracing) or a thin frustum (as in ray differentials). That's really convenient when light propagation has to be tracked beyond primary rays, such as with reflections, shadows, occlusion/visibility rays, for the cones/frustums/differentials can keep growing as the rays bounce around.

However, if we are only concerned with filtering procedural patterns as seen to camera/primary rays, then we can simplify our lives a little bit. Which is what I usually do in my little realtime demos - be lazy and simplify things. So, say one has some basic ray based rendering code that computes a ray for a given pixel, casts it towards the 3D scene, samples the texture of the intersected object and gets ready for shading/lighting. That code might look, perhaps, something like this:

```
// somehow loop over the image plane pixels
vec2 pixel = ...

// generate primary ray for this pixel, and 2 neighbors
Ray ray = calcRayForPixel( pixel );

// compute ray intersection with the scene (return distance, position and normal)
vec3 pos, nor;
intersect( ray, &pos, &nor );

// calc texture coordinates
vec3 uvw = texCoords( pos );

// sample procedural stexture
vec3 col = sampleTexture( uvw );

// perform shading/lighting
...
```

We need to alter this code in order to compute the **uvw** texture coordinate domain that is covered by this pixel undergoing rendering/shading, in order to replace *sampleTexture()* with *sampleTextureWithFilter()*. What we can do is to cast two new rays offset by one pixel up and right, and intersect it against the plane tangent to the surface intersected by the primary ray. Basically, we are trying to see how much did the size of our image plane pixel (which has a real 3D size and orientation in world units) grow as the un-projection of that pixel quad travels with the day across the 3D scene. Constructing the tangent plane is easy, because we have all the information we need: the intersection point of the raycaster, and the normal of the object's surface at the intersection point. Therefore computing the intersection of the two extra corner rays with such plane is nothing but just two divisions, which cost we can consider "for free" really in the context of a raymarcher or a serious raytracer with complex geometry in the scene.

These two extra intersection points, together with the first one, conform a quadrilateral in world space, for which we can get texture coordinates, giving as a quadrilateral in texture coordinate space, much like the GPU does prior to mipmapping. When we have the texture space quadrilateral, which is our filter footprint, we are ready to call *sampleTextureWithFilter()*. So, our code might look something like this:

```
// somehow loop over the image plane pixels
vec2 pixel = ...

// generate primary ray for this pixel, and 2 neighbors
Ray rayC = calcRayForPixel( pixel + vec2(0.0,0.0) );
Ray rayX = calcRayForPixel( pixel + vec2(1.0,0.0) );
Ray rayY = calcRayForPixel( pixel + vec2(0.0,1.0) );

// compute ray intersection with the scene (return distance, position and normal)
vec3 posC, norC;
intersect( rayC, &posC, &norC );

// intersect tangent plane
vec3 posX = rayX.ro - rayX.rd*dot( rayX.ro-posC, norC )/dot( rayX.rd, norC );
vec3 posY = rayY.ro - rayY.rd*dot( rayY.ro-posC, norC )/dot( rayY.rd, norC );

// calc texture coordinates
vec3 uvwC = texCoords( posC );
vec3 uvwX = texCoords( posX );
vec3 uvwY = texCoords( posY );

// sample procedural texture
vec3 col = sampleTextureWithFilter( uvwC, uvwX, uvwY );

// perform shading/lighting
...
```

Conclusion

The trick works remarkably well, and it's really fast (again, in a scenario where lighting, visibility, scene complexity and other factors are the dominant bottlenecks, which is the case more often than not). The benefits of procedural pattern filtering are especially impressive with moving images rather than stills, so here goes a realtime example that implements the technique just described. Feel free to click in the title of the viewport in order to access and inspect the full source code. Also, left-click and move the mouse over the viewport in order to change the fraction of the image that gets filtered/antialiased.



inigo quilez - learning computer graphics since 1994